Defacing

Now that we understand the different types of XSS and various methods of discovering XSS vulnerabilitie learning how to exploit these XSS vulnerabilities. As previously mentioned, the damage and the scope of a type of XSS, a stored XSS being the most critical, while a DOM-based being less so.

One of the most common attacks usually used with stored XSS vulnerabilities is website defacing attacks changing its look for anyone who visits the website. It is very common for hacker groups to deface a webs successfully hacked it, like when hackers defaced the UK National Health Service (NHS) back in 2018. Su echo and may significantly affect a company's investments and share prices, especially for banks and tec

Although many other vulnerabilities may be utilized to achieve the same thing, stored XSS vulnerabilities a vulnerabilities for doing so.

Defacement Elements

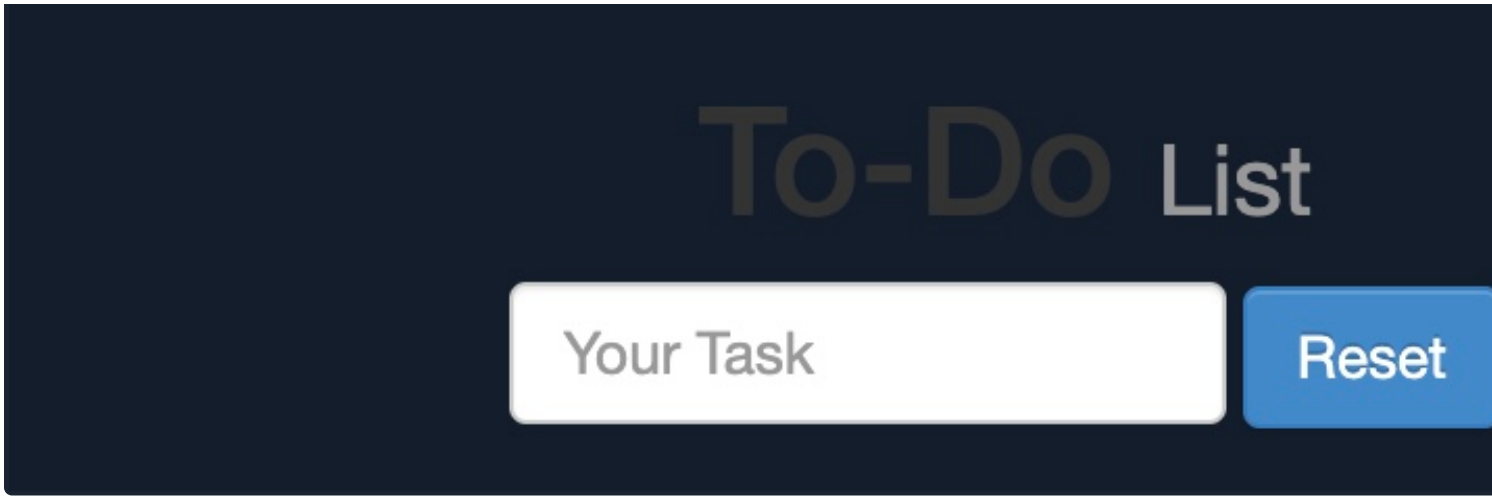
We can utilize injected JavaScript code (through XSS) to make a web page look any way we like. However, used to send a simple message (i.e., we successfully hacked you), so giving the defaced web page a beau target.

Three HTML elements are usually utilized to change the main look of a web page:

- `Background Color` `document.body.style.background`
- `Background` `document.body.background`
- `Page Title` `document.title`
- `Page Text` `DOM.innerHTML`

We can utilize two or three of these elements to write a basic message to the web page and even remove that it would be more difficult to quickly reset the web page, as we will see next.

Changing Background



This will be persistent through page refreshes and will appear for anyone who visits the page, as we are u vulnerability.

Another option would be to set an image to the background using the following payload:

Code: html

```
<script>document.body.background = "https://www.hackthebox.eu/images/logo-htb.svg"</sc
```

Try using the above payload to see how the final result may look.

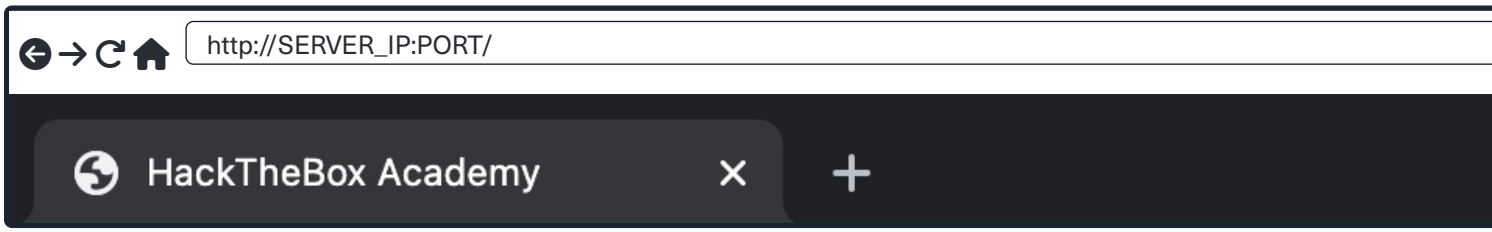
Changing Page Title

We can change the page title from `2Do` to any title of our choosing, using the `document.title` JavaScript f

Code: html

```
<script>document.title = 'HackTheBox Academy'</script>
```

We can see from the page window/tab that our new title has replaced the previous one:



This gives us various options to customize the text on the web page and make minor adjustments to meet groups usually leave a simple message on the web page and leave nothing else on it, we will change the e `body`, using `innerHTML`, as follows:

Code: javascript

```
document.getElementsByTagName('body')[0].innerHTML = "New Text"
```

As we can see, we can specify the `body` element with `document.getElementsByTagName('body')`, and by the first `body` element, which should change the entire text of the web page. We may also use `jQuery` to a before sending our payload and making a permanent change, we should prepare our HTML code separat set our HTML code to the page source.

For our exercise, we will borrow the HTML code from the main page of `Hack The Box Academy`:

Code: html

```
<center>
    <h1 style="color: white">Cyber Security Training</h1>
    <p style="color: white">by
        <img src="https://academy.hackthebox.com/images/logo-htb.svg" height="25px" al
    </p>
</center>
```

Tip: It would be wise to try running our HTML code locally to see how it looks and to ensure that it runs as it in our final payload.

We will minify the HTML code into a single line and add it to our previous XSS payload. The final payload s

Code: html

```
<script>document.getElementsByTagName('body')[0].innerHTML = '<center><h1 style="color
```

Once we add our payload to the vulnerable `To-Do` list, we will see that our HTML code is now permanently

```
<div></div><ul class="list-unstyled" id="todo"><ul>
<script>document.body.style.background = "#141d2b"</script>
</ul><ul><script>document.title = 'HackTheBox Academy'</script>
</ul><ul><script>document.getElementsByTagName('body')[0].innerHTML = '...SNIP...'</sc
</ul></ul>
```

This is because our injected JavaScript code changes the look of the page when it gets executed, which i source code. If our injection was in an element in the middle of the source code, then other scripts or ele so we would have to account for them to get the final look we need.

However, to ordinary users, the page looks defaced and shows our new look.

Previous Next +10 Strea

Cheat Sheet

Table of Contents

XSS Basics

Intro to XSS

Stored XSS

Reflected XSS

DOM XSS

XSS Discovery

XSS Attacks

Defacing

Phishing

Session Hijacking

XSS Prevention

XSS Prevention